

SAiDL 2026 Summer Assignment

Eamon Thomas 2025A3PS1165G

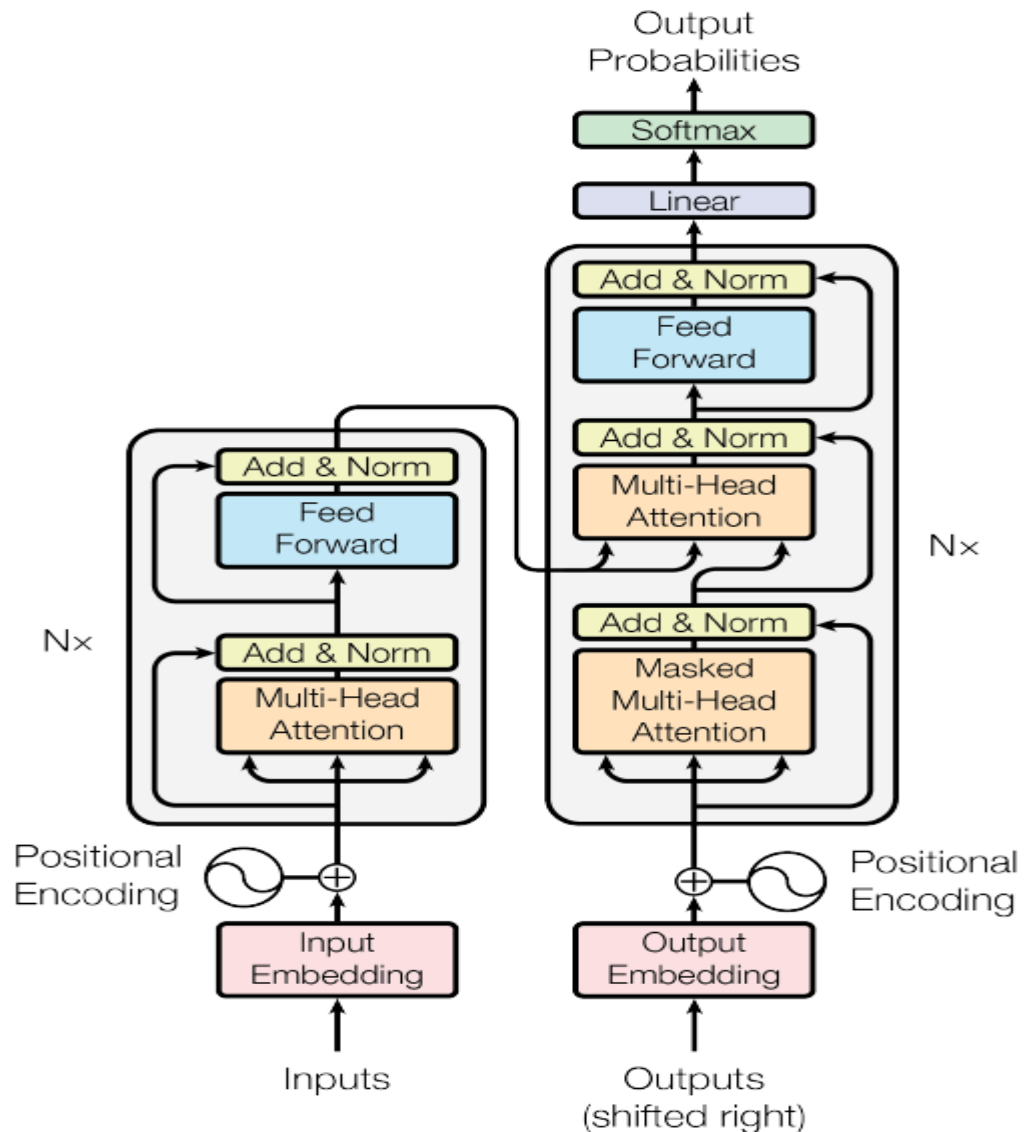
First, we outline the core machine learning task.

Transformers helped us move a step ahead in how we perform NLP (Natural Language Processing). It is a field of research that focuses on giving computers the ability to understand, generate and interpret human language. Older NLP models relied on strict and hand coded linguistic rules. This was later improved by newer models. Next, engineers started using RNNs and LSTMs. They had to read text word-by-word in order, which was slow and made them forget the beginning of long paragraphs. Transformers solved this using a clever trick called "attention." This allows the model to look at an entire sentence all at once and instantly figure out which words are most connected to each other, no matter how far apart they are. It is the core framework behind leading AI technologies, including ChatGPT, translation software, and smart autocomplete tools.

For this assignment, I have built and coded a modular long-context sequence model. I have trained it using the Wikitext - 2 dataset. My base model was a Standard Transformer Model based on the paper - Attention is all you need.

Some basic knowledge on the hyperparameters of this Transformer:

- 1) `batch_size` - number of independent text sequences that are being processed at once in the forward or backward pass
- 2) `block_size` - it is the context window of the transformer. While generating the next token in the sequence the transformer can look at these many tokens that came before it
- 3) `max_iterations` - the transformer undergoes these many iterations and the weights and biases are optimized these many times
- 4) `eval_iteration` - after every `eval_iterations` number of iterations occurs, the transformer pauses its training process and then prints out the training loss and validation loss.



- 5) `learning_rate` - The small change in the weights and biases that happen during the training of the transformer is directly proportional to the value of the learning rate of the transformer. The value of the learning rate should not be too big or too small as it can cause issues.
- 6) `device = 'cuda' if torch.cuda.is_available() else 'cpu'` - Tells the transformer to check if the GPU is available and if yes then use the GPU otherwise use the CPU
- 7) `num_eval_batches` - During evaluation time, the transformer evaluates on

the basis of these many batches

- 8) `n_embedding` - Tells us about the dimensions of the embedding space. Every token is converted to a vector of numbers and the size of the vector depends on the size of the embedding space
- 9) `n_head` - Tells us about the number of heads that our multi-head attention mechanism has. Each head allows us to look at the statement in a new manner.
- 10) `n_layer` - Tells us about the number of attention blocks stacked one upon the other. Each attention block contains feed forward network, multi attention head and layer normalization.
- 11) `dropout` - To prevent overfitting, we randomly turn off a certain percentage of neurons.

I had these hyperparameters

`batch_size = 4`

`block_size = 1024`

`max_iterations = 7000`

`eval_iteration = 200`

`learning_rate = 3e-4`

`device = 'cuda' if torch.cuda.is_available() else 'cpu'`

`num_eval_batches = 100`

`n_embedding = 128`

`n_head = 4`

`n_layer = 4`

`dropout = 0.3`

Basic working of a Transformer: (according to the hyperparameters used)

The model randomly goes in the dataset and grabs 4 random sequences of different lengths. The transformer does not process a single sequence, it processes 4 different sequences in parallel. The sequence can be up to 1024 sequences long and if it is not that long then it is padded and gets cut off if it is longer. Now every token is converted to a list of 128 numbers. This list is the embedding vector which captures the basic meaning of the word. A positional encoding is added to these numbers so the model knows the order of the words. So, now our tokens are in matrix of dimension of (4,1024,128) - (`batch_size`, `block_size`, `n_embedding`)

I have created the transformer in such a way that it has 4 blocks which are stacked one above the other. This data matrix that we have with us right now is

inputted to the first block, then we receive an output and then it is inputted to the next block and so on. The work of the block is to look at how words relate to each other in the sentence, and then process that information.

Each block contains 4 main components

1) Multi-Head Self Attention:

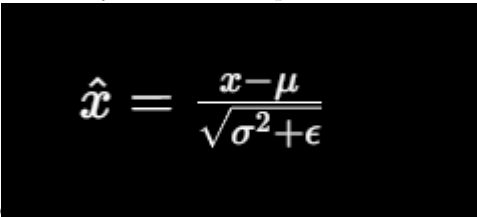
This allows every token to talk to each other and figure out the context. For every token, it calculates a Query, Key and Value vector. The Multi-Head mechanism helps us look at different relationships simultaneously. One head looks at the sequence of tokens with one aspect and the other head with some other aspect. The block splits the 128 dimensional embedding into four 32- dimensional parts.

2) Position-Wise-Feed-Forward Network

After the attention mechanism, the tokens have all the context from the other tokens and they are now sent through a neural network. They are passed through the neural network and then some type of activation function is applied to it after every layer of the neural network.

3) Layer Normalization:

LayerNorm is used right before the data is inputted to the attention layer and before it goes into the Feed-Forward layer. It normalizes the data by calculating the mean and variance of the data and then mean normalization. This is done because when the data is going through the various layers it can become microscopic or just blow up and become massive. Here, we subtract the mean and divide by the variance plus some constant to avoid division by


$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

zero

4) Residual Connections:

It takes the input and then just adds it to the output of the attention mechanism. This happens when we train big transformers, as the model learns the value of the different weights and biases using backpropagation, the value of some gradients starts to become very small (vanishing gradient).

Note: In the Feed-Forward network, I applied dropout. During training, randomly 30% of the activations were set to 0. As a result, the model does not heavily rely on a single neuron.

After passing through all the blocks, the model now tries to predict the next token. Based on the value of the embedding matrix and the values that we get from the blocks, we assign a probability to every token. Now assume there to exist a weighted die that has the same number of sides as that of the number of tokens in our transformer vocabulary. All the sides of the die have a token written on it. If we roll this die, the token with the highest probability will have the highest probability of landing, but this does not eliminate the possibility of other sides landing. As a result, we get the output of the transformer.

This is one iteration, the model repeats this for 7000 such iterations. As per the loss function we get based on these 4 sequences, we adjust the weights and biases which are directly related to the learning rate(0.0003).

final training loss = 4.6881

final validation loss = 5.9645

final test loss = 5.9545

perplexity = $e^{\text{loss}} = 389.5$

total token = iterations x batch size x block size

throughput = total token / time taken

Throughput = 7,850 tokens/sec

Attention heads

Self attention:

Given an input projection matrix X, we derive the linear projections for Queries (Q), Keys (K), and Values (V). The scaled dot-product attention weights are formally computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

To allow the model to jointly attend to information from different representation subspaces at different positions, Multi-Head Attention employs ‘h’ parallel attention heads. Each head utilizes its own set of learned parameter weights. The dimensionality of each head is scaled down to prevent excessive computational overhead. Each independent head computes its attention matrix, and their outputs are subsequently concatenated and mapped via a final linear projection W^O

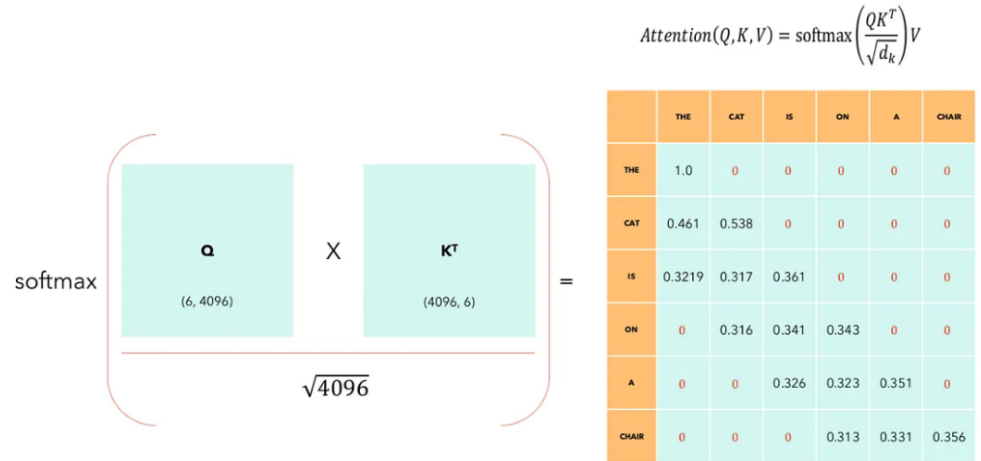
$$d_k = \frac{d_{\text{model}}}{h}$$

Causal masking is applied to the attention score matrix to enforce autoregressive token generation. It replaces subsequent token positions with a large negative value (- infinity) before the softmax layer. This zeroes out attention weights for future context, ensuring predictions at position i rely only on known preceding tokens.

The Variants

1 - Sliding Window/Local Attention

In the original Transformer architecture, the self-attention mechanism has a time complexity of $O(N^2)$, where N is the length of the input sequence. As sequences grow longer (common in natural language processing), the computational cost increases quadratically, making it incredibly expensive to process long texts. Sliding Window Attention reduces this complexity to $O(W \times N)$, where W is a fixed window size. Each token only attends to a small, local window of tokens around it. There is a limitation to this, If the window size is small and the sequence is very long, it requires a massive number of layers for the top layer to see the entire input. .

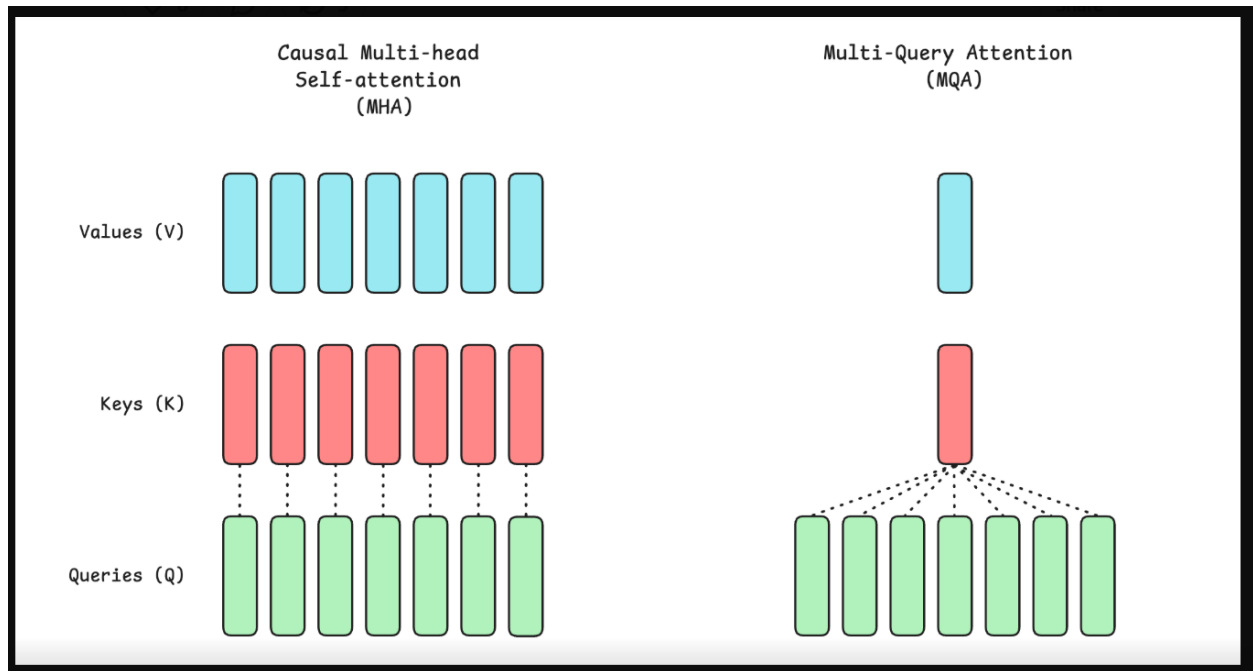


2 - Softmax-free attention (ReLU-based normalization)

This is another attention variant where we change the time complexity from $O(N^2)$ to $O(N)$. The reason why softmax causes the time complexity to be $O(N^2)$ is due to the fact that when we calculate the attention score, we first multiply the query and the transpose of the key matrix. This leads to the creation of a matrix that is $(N \times N)$. But in softmax free attention we first multiply the transpose of the key matrix with the value matrix. This leads to the formation of a matrix whose dimensions are $(N \times d)$ where d is the head dimension i.e. the dimension of the key matrix divided by the number of heads.

3 - Multi-Query Attention (MQA)

In this type of attention mechanism, the number of query heads remain the same but the number of key and value heads are reduced to one. So all the query heads share the same key and value head. As a result the model is able to generate many more tokens at the same time compared to the normal model. But there is a disadvantage that the model loses its ability to look at the dataset from different aspects.



First, a comparison between the different attention variants for a context window of 512 tokens

	Sliding window attention	Softmax free attention	Multi query attention
Train loss	4.7139	5.0496	5.0195
Validation ppl	386.67	410.54	391.02
Test loss	5.9987	6.0106	5.9572
Peak GPU usage	1.54GB	1.48GB	1.93GB
Training Throughput (tokens/sec)	25100.26	23991.44	28220.86
Interference latency(tokens/sec)	90.4532	87.6669	104.4453

Now a comparison between the different attention variants for a context window of 1024 tokens

	Sliding window attention	Softmax Free attention	Multi query attention
Train loss	4.8374	4.7653	4.6983
Validation ppl	387.81	390.80	390.77
Test loss	6.0111	6.0030	5.9817
Peak GPU usage	3.11 GB	2.98GB	3.09GB
Training Throughput(tokens/sec)	28569.81	25922.62	30921.27
Interference la- tency(tokens/sec)	100.5369	97.1813	104.3519

Now a comparison between the different attention variants for a context window of 2048 tokens

	Sliding window attention	Softmax free attention	Multi query attention
Train loss	4.1892	4.3478	4.4160
Validation ppl	451.30	421.63	400.4750
Test loss	6.1678	6.0550	5.9945
Peak GPU usage	7.06 GB	6.56 GB	7.00 GB
Training Throughput(tokens/sec)	21387.56	19441.05	21875.83
Interference la- tency(tokens/sec)	103.7361	96.4961	106.5469

From the tables shown here in this report, I think we can say that **Multi Query Attention** Variant is the best one

Positional Encoding

As the transformer reads all the tokens at once, it does not know about the order of the tokens and hence can easily jumble up the meaning of the sentence by changing the order of the tokens. As a result, we also give the model information about the token order.

$$X = X_{\text{token}} + X_{\text{PE}}$$

$$Q = XW^Q \text{ here } X \text{ is input, } W^Q \text{ is the query weights}$$

$$Q = W^Q X_{\text{PE}} + W^Q X_{\text{token}}$$

But the problem still is that human language is relative. Thus, we now use relative distance between tokens to better understand the context.

- 1) Rotary Positional Encoding (RoPE)

In RoPE, we use vectors. We rotate these vectors while preserving the length of the vector. As a result, we can preserve the meaning of the vector and the position of the vector can be decided using the rotation angle of the vector. Our model uses thousands of dimensions, and these huge vectors are broken into pairs of 2 and each pair is rotated with different speeds.

1. The 2D Rotation Foundation

For a simple two-dimensional vector, the rotation matrix R_{Θ} for a given position m and a base frequency θ is defined as:

$$R_{\Theta, m} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}$$

When applied to a 2D token vector $\mathbf{x} = [x_0, x_1]^T$, the rotated vector $\mathbf{x}'$ is:

$$\mathbf{x}' = R_{\Theta, m} \mathbf{x} = \begin{pmatrix} x_0 \cos(m\theta) - x_1 \sin(m\theta) \\ x_0 \sin(m\theta) + x_1 \cos(m\theta) \end{pmatrix}$$

2) ALiBi (Attention with Linear Biases)

In normal transformers we calculate the attention scores between query at position

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

i and key at position j as

In ALiBi, we calculate the attention score as

$$\text{ALiBi Score} = \frac{QK^T}{\sqrt{d_k}} - m(i - j)$$

We subtract a linear penalty that is directly proportional to the distance between the two tokens.

The value of m:

For the attention score in ALiBi, we use a variable ‘m’. The value of this ‘m’ term depends on the attention head it belongs to. The formula is

$$m = 2^{-\frac{8h}{n}}$$

Here, h is the head index and n is the total number of heads in the layer.

Thus, the value of the penalty is not constant and changes as per the attention head.

3) Relative Position Encoding

Here we calculate the attention score as

$$\text{RPE Score}_{i,j} = \frac{Q_i K_j^T}{\sqrt{d_k}} + b_{c_{i-j}}$$

Here i is query token index

j is the key token index

b is a learnable parameter

Now, we create a distance matrix which has the distance between each token (here we mask the future token distance).

We also limit the value of ci-j; it will always be the minimum of k or (i-j) where k is any random number.

Comparing all the positional embedding variants for a context window of 512 tokens

	ALiBI	RoPE	Relative position embedding
Training loss	5.1232	4.8976	5.1663
Validation ppl	384.3352	375.9502	351.0642
Test loss	5.9767	6.4598	5.9042
Time taken	596.013	731.79	—
Peak GPU	1.72 GB	1.67 GB	1.66 GB
Throughput (tokens/sec)	20034.72	19590.22	20654.09

Comparing all the positional embedding variants for a context window of 1024 tokens

	ALiBi	RoPE	Relative position embedding
Training loss	4.8229	4.5898	4.9029
Validation ppl	372.8439	363.7517	338.81
Test loss	5.9736	6.0489	5.8937
Time taken	938.37	1080.21	-----
GPU	3.4GB	3.33GB	3.23 GB
Throughput (tokens/sec)	23842.93	26543.04	20846.91

Comparing all the positional embedding variants for a context window of 2048 tokens

	ALiBi	RoPE	Relative position embedding
Training loss	4.8454	4.3448	4.6601
Validation ppl	394.3432	358.2668	387.76
Test loss	5.9032	5.8359	5.9631
Time taken	2433.96	2651.68	-----
GPU	7.4392	7.315 GB	7.56 GB
Throughput	17509.34	21625.58	14316.97

From the tables in this report, we can conclude that **RoPE** is the best positional encoding among them

Convolution variant:

Adding a 1D convolution component helps us efficiently capture the local n-gram context. This is useful as the local context is of utmost importance in NLP. With the help of the convolution layer we are able to edit the embeddings of the tokens as per the local context. So now the attention mechanism calculates the attention scores based on these new embeddings. Due to this the scores are better as we get good local and global context and as a result we get better token outputs. The output tokens generated by the 1D convolution layer are the best.

1) Conv1D Layer before each Attention Block:

Before the tokens are converted into Q, K, V, the Conv1D shifts the token embeddings based on their immediate neighbors. The Conv1D smooths out the local noise before global attention takes over. Thus it is a great short term memory buffer.

2) Gated ConvFFN

This is a unique convolution layer where the input vector goes through two path ways. The first one is the normal path. It passes through the linear layers and then a non linear activation function. The second one is a bit different from the first. Here, it goes through the linear layers and then it goes through a convolution layer which emphasizes the local context. Now we have 2 vectors and there occurs element wise multiplication.

	1D Convolution	Conv1D before each layer	Gated ConvFFN
Train loss	3.73	4.28	4.20
Validation ppl	362.47	321.16	303.97
GPU	3.12	3.24	3.27
Throughput	28065	31108	30582

For the final task, I have combined a 1D convolution layer, RoPE and MQA and these were the results

train loss 0.0238, val loss 0.0882

Training Throughput: 28762.44 tokens/sec

Peak GPU Memory: 1.70 GB

Final Test Loss: 0.0890

Final Validation Perplexity: 1.10

Inference Speed: 121.4847 tokens/sec